

Q1. Smart University Payroll Management System

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.Scanner;
abstract class Employee{
    private int employeeId;
    private String employeeName;
    private String department;
    private double basicSalary;
    public Employee(){
    }
    public Employee (int employeeId,String employeeName,String department,double basicSalary){
        this.employeeId=employeeId;
        this.employeeName=employeeName;
        this.department=department;
        this.basicSalary=basicSalary;
    }
    public int getEmployeeId(){
        return employeeId;
    }
    public void setEmployeeId(int employeeId){
        this.employeeId=employeeId;
    }
    public String getEmployeeName(){
        return employeeName;
    }
    public void setEmployeeName(String employeeName){
        this.employeeName=employeeName;
    }
    public String getDepartment(){
        return department;
    }
    public void setDepartment(String department){
        this.department=department;
    }
    public double getBasicSalary(){
        return monthlySalary;
    }
    public void setBasicSalary(double basicSalary){
        this.basicSalary=basicSalary;
    }
    public void displayEmployee(){
        System.out.println("Employee ID : "+EmployeeId);
        System.out.println("Name : "+employeeName);
        System.out.println("Department : "+department);
        System.out.println("Monthly Salary : "+calculateSalary());
    }
    public abstract double calculateSalary();
}
class TeachingFaculty extends Employee{
    private double teachingHours;
    private double incentivePerHour;
    public TeachingFaculty(){
        super();
    }
    public TeachingFaculty(int employeeId,String employeeName,String department,double basicSalary,double teachingHours,double incentivePerHour){
        super(employeeId,employeeName,department,basicSalary);
        this.teachingHours=teachingHours;
        this.incentivePerHour=incentivePerHour;
    }
    public double getTeachingHours(){
        return teachingHours;
    }
}
```

Submissions Report

```

    public void setTeachingHours(double teachingHours){
        this.teachingHours=teachingHours;
    }
    public double getIncentivePerHour(){
        return incentivePerHour;
    }
    public void setIncentivePerHour(double incentivePerHour){
        this.incentivePerHour=incentivePerHour;
    }
    @Override
    public double calculateSalary(){
        return getBasicSalary()+(teachingHours*incentivePerHour);
    }
}

class TechnicalStaff extends Employee{
    private double overtimeHours;
    private double overtimeRate;
    public TechnicalStaff(){
        super();
    }
    public TechnicalStaff(int employeeId,String employeeName,String department,double basicSalary,double overtimeHours,double overtime
Rate){
        super(employeeId,employeeName,department,basicSalary);
        this.overtimeHours=overtimeHours;
        this.overtimeRate=overtimeRate;
    }
    public double getOvertimeHours(){
        return overtimeHours;
    }
    public void setOvertimeHours(double overtimeHours){
        this.overtimeHours=overtimeHours;
    }
    public double getOvertimeRate(){
        return overtimeRate;
    }
    public void setOvertimeRate(double overtimeRate){
        this.overtimeRate=overtimeRate;
    }
    @Override
    public double calculateSalary(){
        return getBasicSalary()+(overtimeHours*overtimeRate);
    }
}

class VisitingFaculty extends Employee{
    private double lecturesHandled;
    private double paymentPerLecture;
    public VisitingFaculty(){
        super();
    }
    public VisitingFaculty(int employeeId,String employeeName,String department,double basicSalary,double lecturesHandled,double payme
ntPerLecture){
        super(employeeId,employeeName,department,basicSalary);
        this.lecturesHandled=lecturesHandled;
        this.paymentPerLecture=paymentPerLecture;
    }
    public double getLecturesHandled(){
        return lecturesHandled;
    }
    public void setLecturesHandled(double lecturesHandled){
        this.lecturesHandled=lecturesHandled;
    }
    public double getPaymentPerLecture(){
        return paymentPerLecture;
    }
}

```

```
public void setPaymentPerLecture(double paymentPerLecture){
    this.paymentPerLecture=paymentPerLecture;
}
@Override
public double calculateSalary(){
    return lecturesHandled*paymentPerLecture;
}
}

public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int n=Integer.parseInt(sc.nextLine().trim());
        Employee[] employees=new Employee[n];
        for(int i=0;i<n;i++){
            String type=sc.nextLine().trim();
            int id=Integer.parseInt(sc.nextLine().trim());
            String name=sc.nextLine().trim();
            String dept=sc.nextLine().trim();
            double basicSalary=Double.parseDouble(sc.nextLine().trim());
            double val1=Double.parseDouble(sc.nextLine().trim());
            double val2=Double.parseDouble(sc.nextLine().trim());
            switch (type){
                case "TeachingFaculty":
                    employees[i]=new TeachingFaculty(id,name,dept,basicSalary,val1,val2);
                    break;
                case "TechnicalStaff":
                    employees[i]=new TechnicalStaff(id,name,dept,basicSalary,val1,val2);
                    break;
                case "VisitingFaculty":
                    employees[i]=new VisitingFaculty(id,name,dept,basicSalary,val1,val2);
                    break;
                default:
                    System.out.println("Unknown employee type: "+type);
            }
        }
        int searchId=0;
        while(sc.hasNextLine()){
            String line=sc.nextLine().trim();
            if(line.isEmpty()){
                continue;
            }
            if(line.matches("-?\\d+")){
                searchId=Integer.parseInt(line);
                break;
            }
        }
        System.out.println("Employee Details");
        for(Employee e : employees){
            e.displayEmployee();
        }
        Employee highest=employees[0];
        double total=0;
        for(Employee e : employees){
            double salary=e.calculateSalary();
            total+=salary;
            if(salary>highest.calculateSalary()){
                highest=e;
            }
        }
        System.out.println("Highest Paid Employee");
        System.out.println("Employee ID : "+highest.getEmployeeId());
        System.out.println("Salary : "+highest.calculateSalary());
        double average=total /employees.length;
        System.out.println("Average Salary");
    }
}
```

```
        System.out.printf("%.2f%n");
        Employee found=null;
        for(Employee e : employees){
            if(e.getEmployeeId()==searchId){
                found=e;
                break;
            }
        }
        if(found!=null){
            System.out.println("Employee Found");
            System.out.println("Employee ID : "+found.getEmployeeId());
            System.out.println("Name : "+found.getEmployeeName());
            System.out.println("Salary : "+found.calculateSalary());
        }else{
            System.out.println("Employee Not Found");
        }
        sc.close();
    }
}
```

INPUT

```
3
TeachingFaculty
101
Dr.Arun
CSE
50000
20
500
TechnicalStaff
102
Mr.Ravi
ECE
35000
15
300
VisitingFaculty
103
Dr.Kumar
IT
0
40
1200
102
```

OUTPUT

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "TeachingFaculty"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:668)
    at java.base/java.lang.Integer.parseInt(Integer.java:786)
    at Employee.main(Employee.java:30)
```

Q2. Smart Vehicle Insurance Claim System

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.Scanner;
abstract class VehicleBase{
    private String vehicleNumber;
    private String ownerName;
    private double insuredAmount;
    public VehicleBase(String vehicleNumber,String ownerName,double insuredAmount){
        this.vehicleNumber=vehicleNumber;
        this.ownerName=ownerName;
        this.insuredAmount=insuredAmount;
    }
    public String getVehicleNumber(){
        return vehicleNumber;
    }
    public String getOwnerName(){
        return ownerName;
    }
    public double getInsuredAmount(){
        return insuredAmount;
    }
    public abstract double calculateClaimAmount();
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int n=Integer.parseInt(sc.nextLine().trim());
        VehicleBase[] vehicles=new VehicleBase[n];
        for(int i=0;i<n;i++){
            String type=sc.nextLine().trim();
            String vehicleNumber=sc.nextLine().trim();
            String ownerName=sc.nextLine().trim();
            double insuredAmount=Double.parseDouble(sc.nextLine().trim());
            switch(type.toLowerCase()){
                case "car":
                    vehicles[i]=new Car(vehicleNumber,ownerName,insuredAmount);
                    break;
                case "bike":
                    vehicles[i]=new Bike(vehicleNumber,ownerName,insuredAmount);
                    break;
                case "truck":
                    vehicles[i]=new Truck(vehicleNumber,ownerName,insuredAmount);
                    break;
            }
        }
        for(VehicleBase v : vehicles){
            System.out.println("Vehicle : "+v.getVehicleNumber());
            System.out.println("Claim Amount : "+v.calculateClaimAmount());
        }
        VehicleBase highest=vehicles[0];
        for(VehicleBase v : vehicles){
            if(v.calculateClaimAmount()>highest.calculateClaimAmount()){
                highest=v;
            }
        }
        System.out.println("Highest Claim Vehicle");
        System.out.println(highest.getVehicleNumber());
        System.out.println(highest.calculateClaimAmount());
        String searchNumber=sc.nextLine().trim();
        boolean found=false;
        for(VehicleBase v : vehicles){
            if(v.getVehicleNumber().equalsIgnoreCase(searchNumber)){
                System.out.println("Vehicle Found");
                System.out.println("Owner : "+v.getOwnerName());
            }
        }
    }
}
```

```
        System.out.print("Claim Amount : "+v.calculateClaimAmount());
        found=true;
        break;
    }
}
if(!found){
    System.out.println("Vehicle Not Found");
}
sc.close();
}
}
class Car extends VehicleBase{
    public Car(String vehicleNumber,String ownerName,double insuredAmount){
        super(vehicleNumber,ownerName,insuredAmount);
    }
    @Override
    public double calculateClaimAmount(){
        return getInsuredAmount()*0.80;
    }
}
class Bike extends VehicleBase{
    public Bike(String vehicleNumber,String ownerName,double insuredAmount){
        super(vehicleNumber,ownerName,insuredAmount);
    }
    @Override
    public double calculateClaimAmount(){
        return getInsuredAmount()*0.60;
    }
}
class Truck extends VehicleBase{
    public Truck(String vehicleNumber,String ownerName,double insuredAmount){
        super(vehicleNumber,ownerName,insuredAmount);
    }
    @Override
    public double calculateClaimAmount(){
        return getInsuredAmount()*0.90;
    }
}
}
```

INPUT

```
3
Car
TN10AB1234
Ramesh
800000
Bike
TN09XY5678
Suresh
100000
Truck
TN11TR9999
Logan
1500000
TN09XY5678
```

OUTPUT

```
Vehicle : TN10AB1234
Claim Amount : 640000.0

Vehicle : TN09XY5678
Claim Amount : 60000.0

Vehicle : TN11TR9999
Claim Amount : 1350000.0

Highest Claim Vehicle
TN11TR9999
1350000.0

Vehicle Found
Owner : Suresh
Claim Amount : 60000.0
```